

Verified Community Directory PWA

Use Case Diagram, Architecture Diagram and Free-Tier Tech Stack Blueprint

Secure MVP Rebuild | Free-Tier Compatible Version 1.1 Draft

Purpose

This document explains how the current Community Directory PWA should be transformed into a secure, backend-driven, installable platform with verified members, trusted-device access, shared Inbox updates, Emergency alerts, admin controls, privacy rules and audit logging.

Document Area	Included Content
Use Case View	Actors, major user goals and UML-style use case diagram
Architecture View	Target system architecture, backend modules and data boundaries
Data Flow View	Secure member enrollment, login, directory access and shared communication flow
Technology View	Free-tier frontend, backend, database, PWA, security, OTP, media, deployment and testing stack
Implementation View	Phased workflow and module ownership for MVP delivery

1. Executive Summary

Architecture principle

The frontend should only present screens and call protected APIs. All identity, access, device, privacy, messaging, moderation and security decisions must be enforced by the backend.

The existing PWA interface is a strong visual baseline, but the secure MVP requires a backend-first architecture. The final system must remove direct public Google Sheet access, browser-generated Member IDs, localStorage-based communication and frontend-only ID checks. The target product is a verified community networking, directory and emergency-response platform.

Current Prototype Limitation	Secure MVP Replacement
Public Google Sheet CSV fetch	Private database served through authenticated backend APIs
Browser-generated Member IDs	Random backend-issued Member IDs
Member ID typed before posting	Author identity derived from secure session
Inbox/Emergency saved in localStorage	Central database-backed posts and alerts
No admin monitoring	Admin dashboard with audit logs, flags and moderation
No trusted-device control	Passkey/WebAuthn + server-side device record + secure cookie

2. Actors and Responsibilities

Actor	Responsibility	Primary Access
Community Member	Search directory, view permitted profiles, post Inbox updates, raise Emergency alerts, request device change	Installable PWA
Administrator	Import/approve members, approve device changes, moderate content, resolve alerts, review flags and audit logs	Admin Dashboard
System Administrator	Configure environment variables, deploy services, manage first secure admin bootstrap, backups and operational settings	Server/deployment environment
Email OTP Provider	Delivers one-time verification codes to registered emails	SMTP integration
Private Media Storage	Stores images/videos securely and returns controlled access URLs	Object storage

3. Use Case Diagram

The use case diagram below shows the secure MVP actors and the main product capabilities. Normal members do not directly add members, approve devices, or moderate content. Admins control official records and security-sensitive operations.

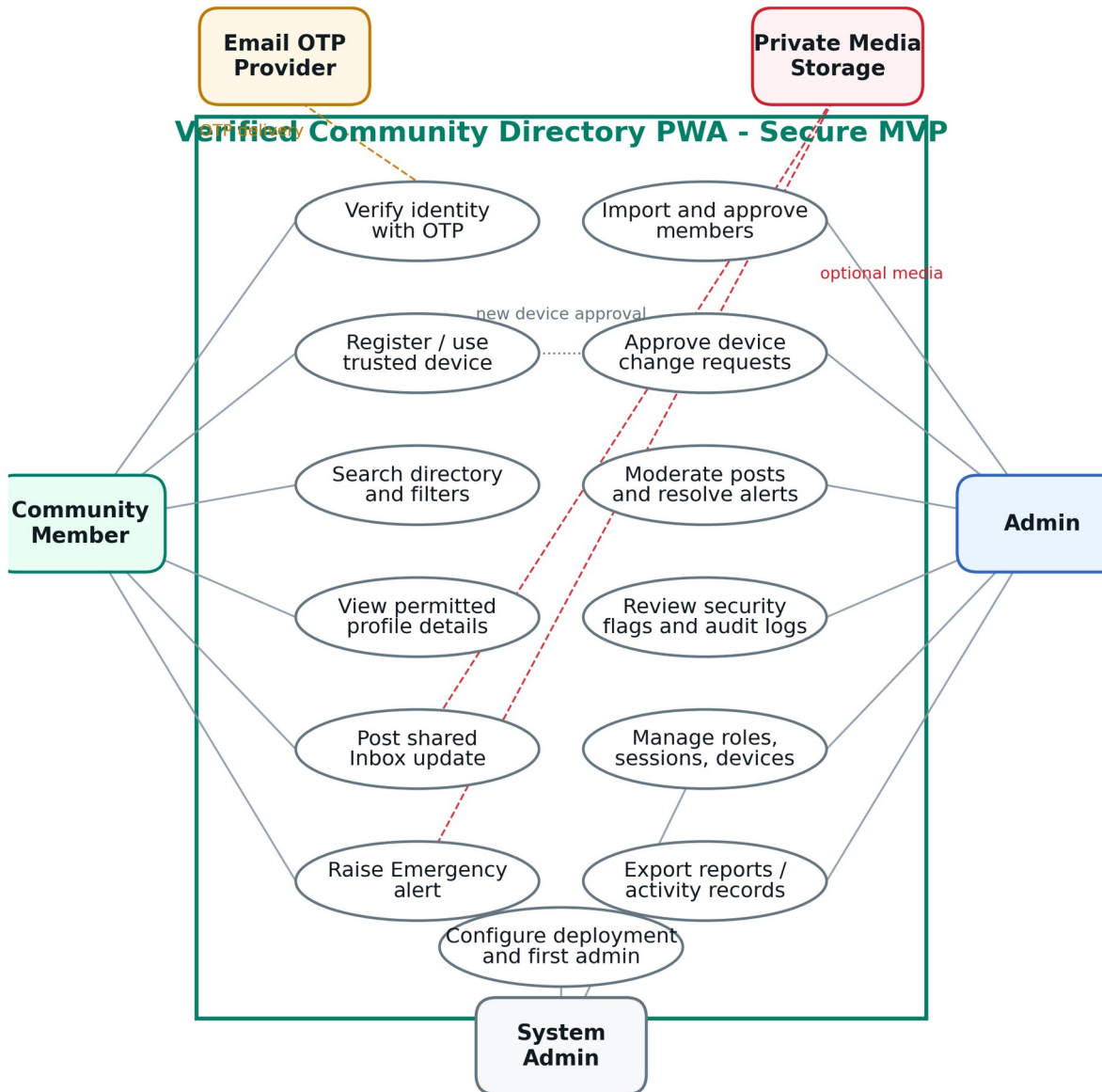


Figure 1: UML-style use case diagram for the Verified Community Directory PWA secure MVP.

3.1 Use Case Summary

Use Case	Primary Actor	Brief Description
Verify identity with OTP	Member	Member proves access to the registered email before enrollment or device-change request.
Register / use trusted device	Member	Member registers a passkey and future access requires WebAuthn assertion plus active device record.
Search directory and filters	Member	Member searches approved directory data returned through privacy-filtered backend APIs.
View permitted profile details	Member	Member sees only fields allowed by role and visibility settings.
Post shared Inbox update	Member	Message is saved centrally and visible to authorised members.
Raise Emergency alert	Member	Urgent alert is stored centrally, highlighted, monitored and resolvable by admins.
Import and approve members	Admin	Admin uploads CSV, reviews staged records and approves official member accounts.
Approve device changes	Admin	Admin approves or rejects pending new-device requests.
Moderate posts and resolve alerts	Admin	Admin removes inappropriate posts and marks emergency alerts as resolved.
Review security flags and audit logs	Admin	Admin investigates failed attempts, suspicious events and admin action history.

4. Target System Architecture Diagram

The architecture keeps the PWA lightweight while shifting all sensitive operations into FastAPI backend modules. The backend acts as the only trusted boundary for data, identity, devices, privacy, moderation and audit decisions.

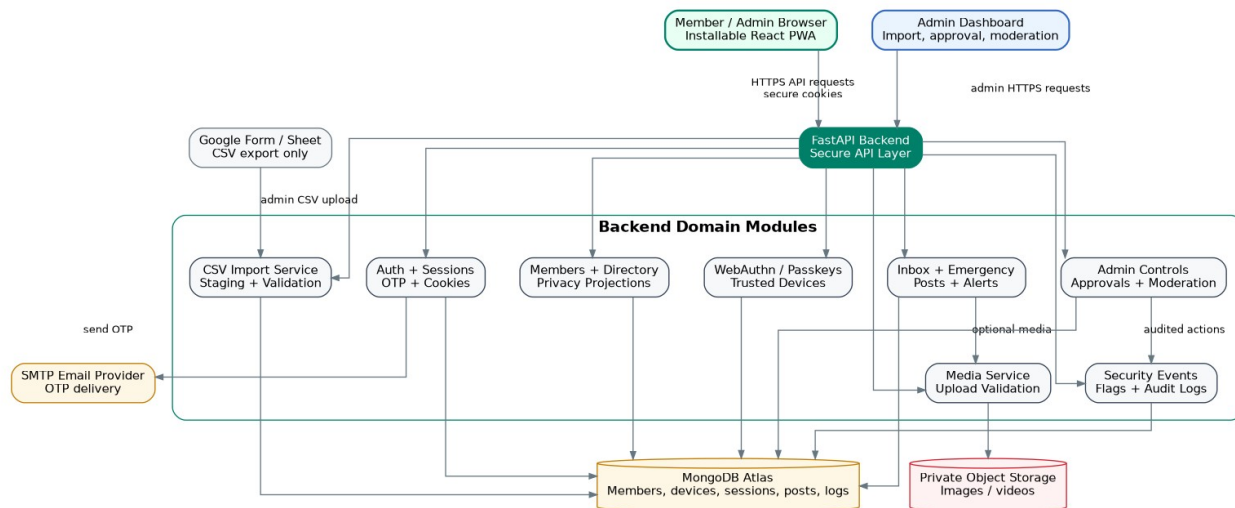


Figure 2: Target secure MVP architecture with PWA, FastAPI backend, MongoDB, SMTP and private object storage.

4.1 Architecture Components

Component	Purpose
React PWA	Mobile-first installable frontend for member directory, profile viewing, Inbox, Emergency and admin screens.
FastAPI Backend	Central API and security layer that validates sessions, devices, roles, requests and data visibility.
Auth + Sessions	Manages OTP challenges, secure cookies, server-side sessions, logout and rate limits.
WebAuthn / Trusted Devices	Manages passkey registration, passkey login, trusted-device cookie and device-change workflow.
Members + Directory	Stores approved profiles, handles search/filter APIs and returns privacy-safe field projections.
Inbox + Emergency	Stores shared posts, urgent alerts, moderation state, media references and alert resolution state.
Admin Controls	Provides member approval, device approvals, post moderation, report export and account suspension.
Security + Audit	Records failed attempts, invalid devices, wrong identity checks, flags and admin action logs.
MongoDB Atlas	Primary data store for members, credentials, devices,

	sessions, posts, flags and audit logs.
Private Object Storage	Stores sensitive images/videos outside public Drive links with controlled backend access.
SMTP Provider	Sends email OTPs to registered member email addresses.

5. Secure Data Flow Diagram

The following diagram separates three important flows: first-time enrollment/login, protected daily usage and the admin control loop. This prevents OTP-only access, typed-ID impersonation and unaudited admin changes.

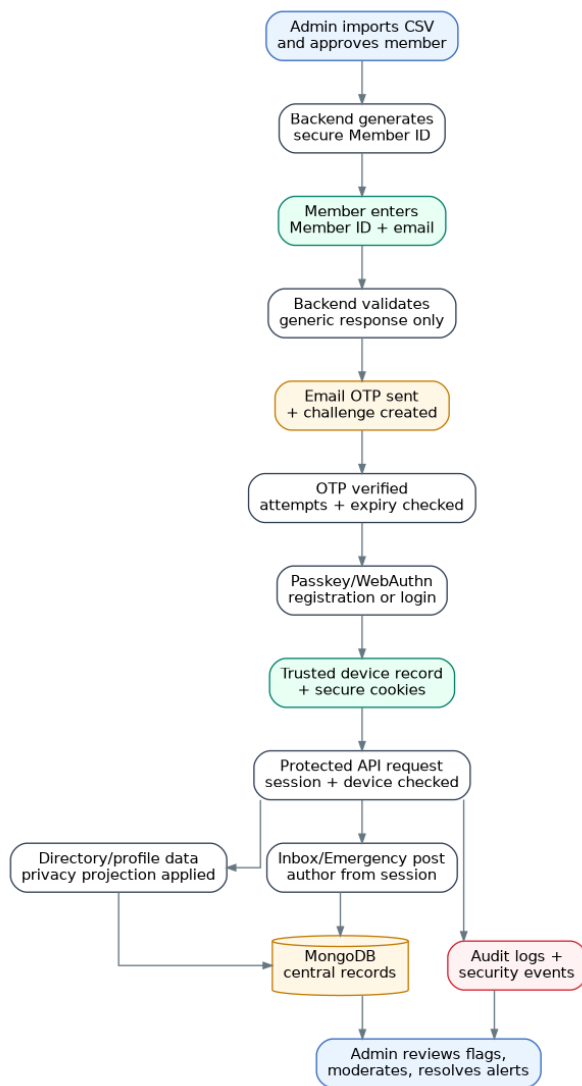


Figure 3: Secure data flow for enrollment, protected usage and admin control loop.

5.1 Member Enrollment Flow

- Admin imports and approves a member record; backend generates a secure Member ID.
- Member enters Member ID and registered email; backend responds generically to prevent enumeration.
- OTP is sent to the registered email only if the submitted details match an approved record.
- After OTP verification, the browser registers a passkey through WebAuthn.
- Backend verifies WebAuthn registration and creates a trusted-device record.
- Backend issues HttpOnly secure session and trusted-device cookies.

5.2 Protected API Request Flow

- PWA sends HTTPS request with secure cookies automatically attached.
- Backend validates session token hash, device token hash, account status and device status.

- Backend applies role and privacy projection rules before returning directory/profile data.
- Inbox and Emergency posts derive author identity from session, never from a typed Member ID.

5.3 Device Change Flow

- New browser or cleared cookie triggers unapproved-device state.
- Member can verify email OTP, but OTP alone cannot activate the new device.
- Backend creates a pending device-change request for admin review.
- After admin approval, old device/sessions are revoked and the new passkey/device is enrolled.

6. Backend and Frontend Module Interaction

The production codebase should be modular. The frontend should not contain data parsing, security decisions, member ID generation or message persistence logic. These belong in backend services and repositories.

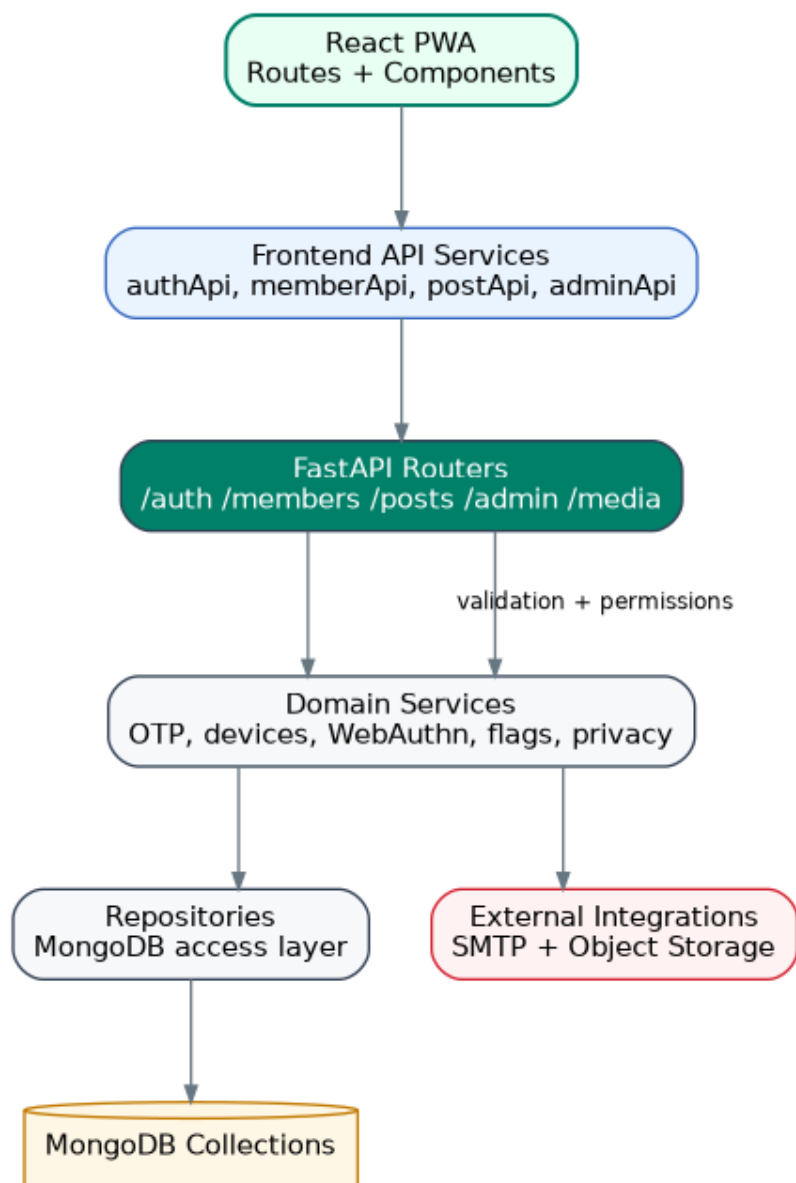


Figure 4: Module interaction from React PWA to FastAPI services and MongoDB repositories.

Layer	Main Modules / Files	Responsibility
Frontend pages	LoginPage, DeviceEnrollmentPage, DirectoryPage, InboxPage, EmergencyPage, AdminDashboard	Route-level screens and layout composition
Frontend components	MemberCard, ProfileSheet, SearchFilters, PostComposer,	Reusable UI pieces migrated from the current visual prototype

	EmergencyAlertCard	
Frontend services	authApi, memberApi, postApi, adminApi, mediaApi	Typed API clients; no local security decisions
Backend routers	auth.py, members.py, posts.py, media.py, admin.py, health.py	HTTP contracts and request validation
Backend services	otp_service, webauthn_service, device_service, privacy_service, flag_service	Business rules and security workflows
Repositories	member_repo, session_repo, post_repo, audit_repo	Database access abstraction and indexes

7. Free-Tier Compatible Tech Stack

Free-tier strategy

The MVP should be designed to run on free-tier-compatible services for pilot/testing. This does not mean the official production system is guaranteed to remain free forever. Free tiers have limits, may sleep, may require billing verification, and should be upgraded when real community usage begins.

Layer	Free-Tier MVP Choice	Why This Fits	Upgrade Trigger
Frontend PWA hosting	Vercel Hobby / Netlify Free / Cloudflare Pages	Static React + Vite PWA can be hosted free with HTTPS, CDN and Git-based deploys.	Upgrade when traffic, team collaboration, advanced analytics or organisation controls are needed.
Frontend framework	React + TypeScript + Vite	Open-source and free. Converts current single-file UI into maintainable components and pages.	No paid upgrade required; complexity may require better engineering discipline, not paid tooling.
PWA support	vite-plugin-pwa + Workbox	Open-source and free. Preserves installable app shell, manifest, icons and controlled caching.	No paid upgrade required; avoid caching private API data.
Backend API hosting	Render Free Web Service for MVP testing OR Google Cloud Run free tier if billing setup is acceptable	Runs FastAPI without maintaining a server. Good enough for demo, internal pilot and validation.	Upgrade for official production, no-sleep backend, SLA needs, higher traffic or sensitive community reliability.
Backend framework	Python FastAPI	Open-source and free. Strong fit for secure APIs, validation, modular routes and async services.	No paid upgrade required.
Database	MongoDB Atlas Free M0 cluster	Free small-scale managed MongoDB suitable for pilot members, sessions, posts and audit logs.	Upgrade when storage, operations, backups, performance or production observability limits are hit.
OTP provider	Resend Free transactional email OR Gmail/SMTP for development only	Email OTP avoids SMS cost and can work within free email limits for a controlled pilot.	Upgrade if daily/monthly OTP volume increases, custom domain sending reputation is needed, or SMS OTP becomes mandatory.
Trusted device	WebAuthn / Passkeys using browser APIs + backend library	No vendor fee for core passkey flow. Correct PWA-compatible alternative to IMEI.	No paid upgrade required; may need stronger domain/certificate planning for production.
Session security	Server-side opaque	Implemented in backend	Consider managed identity

	sessions + HttpOnly Secure cookies	and database. No paid identity provider required for MVP.	only if admin/user scale becomes complex.
Media storage	Cloudflare R2 Free Tier with strict upload limits	Private object storage can support limited image/video attachments without public Drive links.	Upgrade if media exceeds free storage/operation limits or needs lifecycle/backups.
Admin dashboard	Same React PWA with admin routes	No separate paid admin product needed. Admin screens share frontend/backend infrastructure.	Upgrade only if advanced admin analytics or role governance is required.
Logs and audit	MongoDB collections + structured app logs	Free within database limits and enough for MVP accountability.	Upgrade to external logging/monitoring when production debugging and retention become critical.
Testing	Pytest + Playwright	Open-source and free. Covers API, security and end-to-end flows.	No paid upgrade required unless using hosted CI at scale.

7.1 Free-Tier MVP Deployment Setup

Concern	Free-Tier MVP Option	Notes / Limits
Frontend hosting	Vercel Hobby, Netlify Free or Cloudflare Pages	Use HTTPS. Keep frontend static. Do not expose Sheet URLs, secrets or real fallback member data.
Backend hosting	Render Free Web Service for demo/pilot; Cloud Run free tier as alternative	Render free services are suitable for testing/hobby usage, not official production. Cloud Run free tier may require billing account setup.
Database	MongoDB Atlas Free M0	Small-scale development/pilot cluster. Add indexes and keep media out of MongoDB. Upgrade before official production load.
OTP email	Resend Free or SMTP provider	Email OTP only for MVP. Avoid SMS until budget is approved. Enforce resend cooldown and daily caps.
Media	Cloudflare R2 Free Tier	Use strict file-size limits. Prefer images first; videos can exhaust free limits quickly.
Passkeys	Browser WebAuthn + backend verification	No external paid identity provider required. Final domain should be planned early because WebAuthn

		depends on origin/RP ID.
Domain	Free platform subdomain for MVP	Custom domain is optional for pilot but recommended for official launch; custom domains usually cost money.
Backups	Manual export during MVP	Free database tier may not provide full production backup features. Use exports until paid backup is justified.

7.2 Free-Tier Limits and Upgrade Triggers

Area	Free-Tier Reality	When to Upgrade
Backend reliability	Free backend instances may sleep, restart, or have limited resources.	Upgrade before official community rollout or any emergency-critical usage.
Database capacity	Free database clusters are intended for small-scale development/pilot usage.	Upgrade when member count, posts, logs or query load increase.
Email OTP volume	Free email providers cap daily/monthly emails.	Upgrade when OTP traffic crosses free limits or delivery reputation matters.
Media uploads	Free object storage is generous for images but not unlimited, especially with videos.	Upgrade when storage/operations exceed limits or retention becomes important.
Custom domain	Free platform subdomains work for MVP but are not ideal for trust and WebAuthn planning.	Buy domain before wider pilot/official launch.
SMS OTP	Not included in free MVP. SMS OTP usually becomes paid.	Add only if community officially requires phone OTP and budget exists.

Recommended wording for seniors/admins

The proposed system is free-tier compatible for MVP and pilot deployment. For official production use, the architecture is ready to scale to paid tiers when reliability, capacity, SMS OTP, custom domain, backups or media volume require it.

7.3 Cost-Safe MVP Rules

- Use email OTP first; do not promise free SMS OTP.
- Keep video uploads disabled or strictly limited in early pilot.
- Do not store media files inside MongoDB; use object storage.
- Use a free platform subdomain for MVP, but plan a custom domain before official launch.
- Use polling instead of WebSockets in MVP to reduce backend complexity and cost.
- Avoid third-party paid authentication providers; implement sessions and WebAuthn in backend.
- Keep admin analytics simple: audit tables and filters instead of paid dashboards.

8. Main Data Stores and Collections

Collection	Purpose	Critical Indexes
members	Approved member profiles, status, role, visibility settings and secure Member ID	member_id unique, email unique
devices	Trusted device records, device status, trusted-cookie hash and approval history	device token hash unique, member_id
webauthn_credentials	Credential ID, public key, sign counter and linked member/device	credential_id unique
sessions	Server-side active/revoked sessions with expiry	session token hash unique, expires_at TTL
otp_challenges	Hashed OTP, attempts, expiry, resend cooldown and purpose	expires_at TTL, member_id
posts	Inbox messages and Emergency alerts using type field	type + created_at
media_objects	Private media metadata, owner, content type and storage key	owner_member_id, created_at
security_events	Wrong IDs, invalid OTPs, invalid devices, suspicious API calls	event_type, created_at
security_flags	Admin-reviewable aggregated suspicious behaviour	status, created_at
audit_logs	Admin changes, moderation actions, device approvals and revocations	actor_id, target_id, created_at
import_runs	CSV import status, row validation errors and approval metadata	created_at, status

9. API Surface Overview

Module	Representative Endpoints	Security Rule
Auth	POST /auth/identify, POST /auth/otp/verify, GET /auth/me, POST /auth/logout	Generic error responses, OTP limits, secure sessions
WebAuthn	POST /webauthn/register/options, /verify, /login/options, /login/verify	Validate challenge, origin, RP ID, credential and account status

Members	GET /members, GET /members/{member_id}, GET /members/filters	Active session + active trusted device + privacy projection
Posts	GET /posts, POST /posts, POST /posts/{id}/report	Author from session; no typed Member ID identity
Emergency	GET /emergency-alerts, POST /emergency-alerts, POST /emergency-alerts/{id}/resolve	Members create, admins resolve
Media	POST /media/upload-intent, POST /media/complete, GET /media/{id}/access-url	Validate file size/type and ownership
Admin	POST /admin/imports/members, PATCH /admin/members/{id}, POST /admin/device-change-requests/{id}/approve	Admin role required; audit log mandatory

10. Implementation Workflow

Phase	Goal	Deliverables
Phase 0 - Legacy Freeze	Stop extending the insecure single-file prototype	Keep UI reference, remove real fallback data, document screens and fields
Phase 1 - Backend Foundation	Create secure API and database base	FastAPI app, MongoDB connection, config, health, audit utility, admin bootstrap
Phase 2 - Admin Import	Replace public Sheet as live source	CSV import, staging, validation, approval, backend Member ID generation
Phase 3 - Protected Directory	Connect PWA to backend directory APIs	React components, search/filter APIs, profile privacy projection
Phase 4 - OTP and Sessions	Introduce verified member access	Email OTP, hashed challenges, secure cookies, logout, rate limits
Phase 5 - Trusted Devices	Enforce approved-device policy	WebAuthn registration/login, device cookies, device-change approval
Phase 6 - Shared Communication	Replace localStorage demos	Database-backed Inbox, Emergency alerts, media upload, admin moderation
Phase 7 - Monitoring	Add accountability and investigation tools	Security events, flags, audit viewer, exports, suspension/revocation
Phase 8 - PWA Hardening	Prepare pilot deployment	Safe service worker, HTTPS, E2E tests, backups, documentation

11. Non-Negotiable Architecture Rules

- No public Google Sheet CSV endpoint in frontend production code.
- No real personal member data inside frontend source code or fallback arrays.
- No browser-generated Member IDs.
- No typed Member ID as proof of identity.
- No localStorage-based shared Inbox or Emergency persistence.
- No public Google Drive links for sensitive media attachments.
- No IMEI-based device strategy for the PWA.
- No OTP-only activation for a new device.
- No frontend-only authorization or role checks.
- No raw OTPs, passwords, session tokens or cookie values in logs.
- No admin approval, revocation or moderation action without audit log.

12. Architecture Acceptance Criteria

Check	Pass Condition
Directory access	Unauthenticated users and unapproved devices receive no member data.
Identity	A user cannot post as someone else by typing another Member ID.
Messaging	Inbox/Emergency posts appear on other verified accounts because they are database-backed.
Device security	A new browser or cleared cookie requires OTP and admin-approved device change.
Admin controls	Admins can import/approve members, moderate posts, resolve alerts and revoke devices.
Privacy	Profile APIs return different fields for member and admin roles.
Audit	Security-sensitive admin actions produce audit records.
PWA	App shell is installable, but sensitive API responses are not blindly cached.

13. Appendix: Recommended Folder Structure

```
community-directory/  
├── frontend/  
│   ├── src/  
│   │   ├── pages/  
│   │   ├── components/  
│   │   ├── services/  
│   │   ├── hooks/  
│   │   └── pwa/  
│   └── package.json  
├── backend/  
│   ├── app/  
│   │   ├── api/  
│   │   ├── core/  
│   │   ├── models/  
│   │   ├── services/  
│   │   ├── repositories/  
│   │   └── main.py  
│   ├── tests/  
│   └── Dockerfile  
├── legacy-ui-reference/  
├── docs/  
├── docker-compose.yml  
└── README.md
```